

Stack ou pas Stack — Règles d'initiation rapide — variante quadratique

Version d'initiation issue de Neon Stack V5.4 harmonisée, avec remplacement du bonus Fibonacci par un bonus quadratique et casse au 7e cadre.

Cette version vise des parties plus courtes et plus lisibles :

- objectif abaissé à **11 points** ;
- mémoire de départ abaissée à **11 mémoire totale / 11 mémoire libre** ;
- decks symétriques de **26 cartes** ;
- conservation de la phase de mise à jour automatique, des piles d'appels, des cadres parasites, du reboot et du bonus quadratique.

1. Idée générale

Chaque joueur construit un programme en lançant, en faisant évoluer, puis en terminant des fonctions.

Le jeu repose sur :

- la mémoire ;
- les piles d'appels ;
- les fonctions cassées ;
- les cadres parasites ;
- les effets de récursion.

Il n'y a pas de combat entre unités. La partie se joue sur la gestion de la mémoire, le bon moment pour lancer une fonction, la protection de ses piles et la perturbation des fonctions adverses.

2. Condition de victoire

Un joueur gagne immédiatement s'il atteint **11 points ou plus**, à condition d'avoir terminé :

- au moins **3 fonctions** ;
- avec au moins **2 noms de fonctions différents**.

Un joueur conserve toujours ses points.

S'il atteint 11 points ou plus sans remplir encore les autres conditions, il ne gagne pas immédiatement. Il gagne dès qu'une terminaison ou un effet ultérieur le laisse à 11 points ou plus tout en remplissant aussi ces conditions.

Variante stricte : pour une partie moins rapide, conserver la condition V5.4 complète : 4 fonctions terminées avec 3 noms différents.

3. Mise en place

Chaque joueur prend un deck d'initiation de **26 cartes**.

Chaque joueur sépare son deck en deux piles :

- **pile Fonctions** : uniquement les cartes Fonction ;
- **pile Système** : les cartes Commande, Interrupt et Hardware.

Ces deux piles sont mélangées séparément.

Main de départ

Chaque joueur pioche **5 cartes** au total. Répartition conseillée :

- **3 cartes** depuis la pile Fonctions ;

- **2 cartes** depuis la pile Système.

Premier joueur

Le premier joueur est désigné aléatoirement.

Le premier joueur **ne pioche pas** à son premier tour.

Mulligan

Chaque joueur peut effectuer **un mulligan une seule fois** au début de la partie :

- il remélange sa main dans ses deux piles ;
- il repioche 5 cartes selon la répartition de son choix ;
- puis il place 1 carte de sa main sous la pile de son choix.

4. Mémoire

Chaque joueur commence avec :

- **11 mémoire totale** ;
- **11 mémoire libre**.

La mémoire libre sert à payer :

- les cartes ;
- les appels récurifs supplémentaires ;
- certains effets demandant explicitement un paiement en mémoire.

La mémoire totale fixe la quantité maximale de mémoire qu'un joueur peut posséder.

On appelle **mémoire utilisée** la somme de :

- la mémoire réservée par les Fonctions actives ou cassées ;
- la mémoire occupée par les Hardware actifs.

Quand la mémoire totale baisse, la mémoire libre ne peut jamais rester supérieure à cette nouvelle mémoire totale. Si c'est le cas, elle est réduite à cette nouvelle valeur.

5. Types de cartes

Il existe quatre types de cartes :

- **Fonction** : carte principale du jeu. Elle crée une pile d'appels, produit des points et des effets.
- **Commande** : carte jouée pendant ton tour.
- **Interrupt** : carte jouée en réaction pendant n'importe quel tour, si sa condition est remplie et si son coût peut être payé.
- **Hardware** : amélioration durable qui reste en jeu.

Le coût imprimé sur une carte est payé en mémoire libre, sauf mention contraire.

6. Paiement et libération de la mémoire

Quand une carte est jouée, son coût est payé immédiatement en mémoire libre.

- Une **Commande** libère sa mémoire dès que son effet est entièrement résolu.
- Un **Interrupt** libère sa mémoire dès que son effet est entièrement résolu.
- Un **Hardware** conserve sa mémoire occupée tant qu'il reste en jeu.
- Une **Fonction** conserve sa mémoire initiale réservée tant qu'elle est active, cassée, réparée ou en cours d'exécution.

La mémoire libre ne peut jamais descendre sous 0. Si un effet fait perdre plus de mémoire libre que le joueur n'en possède, elle devient simplement 0.

7. Limites en jeu

Un joueur peut contrôler au maximum :

- **3 fonctions actives** ;
- **2 Hardware actifs**.

Un joueur ne peut pas jouer une Fonction s'il contrôle déjà 3 fonctions actives.

Une Fonction active ne peut pas être défaussée volontairement tant qu'elle n'est pas terminée ou cassée, sauf effet de carte.

Si un joueur joue un Hardware alors qu'il en contrôle déjà 2, il choisit immédiatement un de ses Hardware actifs et le met en défausse.

8. Coût mémoire des fonctions

Quand une fonction est lancée :

- son coût mémoire initial est payé en mémoire libre ;
- cette mémoire devient réservée ;
- son cadre initial est placé dans sa pile ;
- cette mémoire reste occupée tant que la fonction n'est pas terminée, nettoyée, supprimée par effet, ou réparée selon le texte applicable.

Chaque fois qu'une fonction reçoit un cadre récursif supplémentaire, son contrôleur doit payer **1 mémoire libre**, sauf si une carte modifie ce coût.

Quand un cadre récursif normal est dépilé, la mémoire correspondant à cet appel est libérée.

Quand le cadre initial est dépilé, la mémoire initiale réservée par la fonction est libérée.

Les cadres parasites ne libèrent jamais de mémoire.

9. Phase de mise à jour des fonctions

Au début du tour d'un joueur, toutes ses fonctions actives non cassées passent par une phase de mise à jour, une par une.

Le joueur actif choisit l'ordre dans lequel ses fonctions actives non cassées sont mises à jour.

- Si une fonction n'a pas encore atteint **[0]**, elle empile automatiquement un nouveau cadre.
- Si une fonction a déjà atteint **[0]**, elle dépile automatiquement son cadre du sommet.

Le joueur ne choisit pas entre empiler et dépiler pendant cette phase. L'état de chaque fonction détermine son évolution.

10. Empiler et dépiler

Empiler X

Une fonction avec **Empiler X** a une profondeur cible fixe de X.

Quand elle est lancée, on place son cadre initial **[X]**.

Puis, à chaque mise à jour, si elle n'a pas encore atteint **[0]**, elle reçoit successivement **[X-1]**, **[X-2]**, etc., jusqu'à **[0]**.

Empiler jusqu'à X

Une fonction avec **Empiler jusqu'à X** permet au joueur de choisir, au lancement, une profondeur cible comprise entre 0 et X.

Le cadre initial porte cette valeur choisie.

Dépiler

Dépiler consiste à retirer le cadre du sommet d'une pile et à appliquer son effet selon sa nature.

11. Atteindre [0], puis remonter

Une fonction ne peut pas dépiler tant qu'elle n'a pas atteint son cadre [0].

Une fois le cadre [0] placé, la fonction entre en phase de remontée.

Une fonction en phase de remontée ne peut plus recevoir d'empilage normal. Seuls les effets de cartes peuvent encore lui ajouter des cadres après cela.

12. Cas de base, remontée, terminaison

Le cas de base est le cadre [0].

Il est :

- le dernier cadre empilé normalement ;
- le premier cadre dépilé en remontée.

Une fonction n'est pas terminée quand son cas de base est résolu.

Elle est terminée seulement quand son cadre initial est dépilé à son tour et que sa pile devient vide.

Chaque fonction peut donc avoir jusqu'à trois niveaux d'effet :

- **Cas de base** : effet du cadre [0] ;
- **Remontée** : effet des cadres supérieurs à [0] ;
- **Terminaison** : effet final, appliqué seulement quand la pile de la fonction devient vide.

13. Valeur de récursion et bonus quadratique

On note **R** la valeur de récursion de la fonction.

La valeur de R est fixée au lancement de la fonction :

- si la carte dit **Empiler X**, alors $R = X$;
- si la carte dit **Empiler jusqu'à X**, alors R est égal à la profondeur choisie au lancement.

Les cadres parasites ne modifient jamais R.

Les cadres ajoutés par un effet de carte ne modifient pas R, sauf mention explicite du contraire.

Bonus quadratique de jeu utilisé :

R	0	1	2	3	4	5
$B(R)=R^2$	0	1	4	9	16	25

Par convention de jeu, **$B(R)=R^2$** : une fonction lancée sans appel récursif ne reçoit pas de bonus.

14. Score d'une fonction terminée

La valeur imprimée sur une carte Fonction correspond à ses points de base.

Quand une fonction se termine :

1. son contrôleur marque ses points de base ;
2. il ajoute un bonus de récursion égal à $B(R)$;
3. il résout l'effet de Terminaison ;
4. la fonction est défaussée ;
5. toute la mémoire qu'elle occupait est libérée.

15. Fonctions cassées

Une fonction casse si :

- elle devrait empiler mais son contrôleur n'a pas assez de mémoire libre pour payer cet appel ;
- elle devrait recevoir un cadre au-delà de la limite autorisée ;
- un effet de carte la casse explicitement.

Une fonction cassée :

- reste en jeu ;
- ne peut plus être empilée ni dépilée ;
- ne rapporte aucun point ;
- continue d'occuper toute la mémoire déjà engagée ;
- compte toujours dans la limite des fonctions actives.

16. Nettoyer et réparer

Nettoyer une fonction cassée consiste à la défausser. Toute sa mémoire est alors libérée.

Réparer une fonction cassée consiste à la remettre en état selon le texte de la carte utilisée.

Par défaut, une fonction réparée revient avec :

- sa mémoire initiale toujours réservée ;
- son cadre initial seulement ;
- aucun cadre parasite, sauf mention contraire.

17. Reboot

À la place de toute autre action pendant son tour, un joueur peut effectuer un reboot.

Dans ce cas :

- toutes ses fonctions actives, intactes ou cassées, sont défaussées ;
- toutes leurs piles sont retirées sans produire d'effet ;
- il récupère immédiatement toute la mémoire occupée par ses fonctions ;
- ses Hardware restent en jeu et continuent d'occuper leur mémoire ;
- il défausse toute sa main ;
- il mélange sa défausse dans ses deux piles de pioche, en respectant les types ;
- les cartes Fonction retournent dans la pile Fonctions ;
- les cartes Commande, Interrupt et Hardware retournent dans la pile Système ;
- il pioche 5 nouvelles cartes, si possible ;
- il ne peut effectuer aucune autre action ce tour-ci.

18. Cadres parasites et overflow

Un cadre parasite :

- compte pour la taille de la pile ;
- doit être dépilé normalement ;
- n'applique aucun Cas de base, aucune Remontée, aucune Terminaison ;
- ne libère pas de mémoire lorsqu'il est dépilé ;
- ne modifie jamais R.

Un cadre parasite ajouté avant que la fonction atteigne **[0]** ne bloque pas la descente. La fonction continue d'empiler les cadres récursifs attendus jusqu'à atteindre **[0]**. Les cadres parasites seront dépilés pendant la remontée lorsqu'ils arriveront au sommet.

Une pile ne peut contenir que **6 cadres**.

Si un effet ou une mise à jour devrait ajouter un **7e cadre**, il y a overflow : la fonction casse aussitôt, les ajouts restants ne sont pas appliqués et la pile n'est plus résolue normalement ensuite.

19. Épuisement de pioche et perte de mémoire totale

Si un joueur doit piocher une carte et que ses deux piles sont vides, il ne pioche pas et perd **1 mémoire totale**.

Après cette perte :

1. si sa mémoire libre est supérieure à sa nouvelle mémoire totale, elle est réduite à cette nouvelle valeur ;
2. si sa mémoire utilisée dépasse sa mémoire totale actuelle, il effectue immédiatement un reboot forcé ;
3. si sa mémoire totale tombe à 0, il perd immédiatement la partie.

La pioche B(R) n'est pas plafonnée.

20. Interrupts et ordre de réaction

Les Interrupts peuvent être joués pendant n'importe quel tour, dès que leur condition de déclenchement est remplie et que leur coût peut être payé.

Quand un événement déclencheur se produit :

1. le joueur non actif a la première possibilité de jouer un Interrupt ;
2. puis les joueurs alternent, un Interrupt à la fois ;
3. les Interrupts se résolvent ensuite dans l'ordre inverse de leur pose ;
4. un joueur qui veut répondre à un Interrupt doit le faire immédiatement ;
5. un joueur ne peut pas attendre puis revenir plus tard dans la même séquence.

Si les deux joueurs passent successivement, la séquence de réaction s'achève et le jeu reprend.

21. Déroulement d'un tour

1. **Phase de mise à jour** : le joueur actif choisit l'ordre de mise à jour de ses fonctions actives non cassées.
2. **Pioche** : le joueur pioche 1 carte dans la pile de son choix. Le premier joueur ne pioche pas à son premier tour. Si les deux piles sont vides au moment où il doit piocher, il applique la règle d'épuisement de pioche.
3. **Conception** : le joueur peut jouer des Fonctions, des Commandes et des Hardware tant qu'il peut payer leur coût, ou choisir d'effectuer un reboot à la place de toute autre action.
4. **Fin de tour** : les effets temporaires cessent ; la mémoire libre temporaire non dépensée disparaît.

22. Intention pédagogique

Cette version doit faire sentir que :

- une fonction descend jusqu'au cas de base puis remonte ;
- une récursion profonde rapporte beaucoup plus, mais expose davantage à l'overflow ;
- la mémoire engagée bloque les choix futurs ;
- une fonction cassée est une fuite durable ;
- les cadres parasites ralentissent ou sabotent l'exécution ;
- réparer, nettoyer ou accélérer une fonction sont des décisions tactiques lisibles.